

Caso práctico

Esta mañana **Juan** ha llegado muy temprano a la empresa. Hay que dar una solución eficiente a la aplicación corporativa de la asociación de periodistas XXL, una aplicación que según cuentan los clientes se ha vuelto muy perezosa, muy lenta, y constantemente se bloquea, o da la impresión de que se bloquea.

La aplicación, entre otras cosas proporciona diferentes servicios en red, como impresión, almacenamiento y distribución de archivos, así como mensajería interna. También gestiona su propio servidor web interno.

Esta aplicación, se diseñó para una Intranet de no más de 5 usuarios, y en la actualidad la asociación cuenta con más del doble; además se ha creado una nueva sucursal que se interconectará en breve con la sede principal.

Juan le ha estado dando vueltas, y después de contrastar y consensuar opiniones con **Ada**, **María** y **Ana**, parece que ya lo tiene algo más claro: ¿soportarán algunos de los servicios programados en la aplicación comunicaciones simultáneas? Todos apuestan a que no.

Por otra parte, ya puestos a revisar código, la asociación ha encargado nuevas funcionalidades para su aplicación en red, por lo que **Juan** anda muy liado buscando sus apuntes de un Máster que realizó, no hace mucho, sobre generación de servicios en red con Java. Quiere comenzar con los nuevos servicios.



[Ministerio de Educación y Formación Profesional](#) (Dominio público)

Materiales formativos de FP Online propiedad del Ministerio y Formación Profesional.

[Aviso Legal](#)

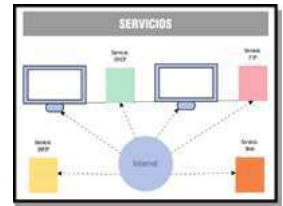
1.- Introducción.

Una red de ordenadores o red informática la podemos definir como un sistema de comunicaciones que conecta ordenadores y otros equipos informáticos entre sí, con la finalidad de compartir información y recursos.

Mediante la compartición de información y recursos en una red, los usuarios de la misma pueden hacer un mejor uso de ellos y, por tanto, mejorar el rendimiento global del sistema u organización.

Las **principales ventajas de utilizar una red de ordenadores** las podemos resumir en las siguientes:

- Reducción en el presupuesto para software y hardware.
- Posibilidad de organizar grupos de trabajo.
- Mejoras en la administración de los equipos y las aplicaciones.
- Mejoras en la integridad de los datos.
- Mayor seguridad para acceder a la información.
- Mayor facilidad de comunicación.



Pues bien, **los servicios en red son responsables directos de estas ventajas.**

Un servicio en red es un software o programa que proporciona una determinada funcionalidad o utilidad al sistema. Por lo general, estos programas están basados en un conjunto de protocolos y estándares

Una **clasificación de los servicios en red** atendiendo a su finalidad o propósito puede ser la siguiente:

- ✓ **Administración/Configuración.** Esta clase de servicios facilita la administración y gestión de las configuraciones de los distintos equipos de la red, por ejemplo: los servicios DHCP y DNS.
- ✓ **Acceso y control remoto.** Los servicios de acceso y control remoto, se encargan de permitir la conexión de usuarios a la red desde lugares remotos, verificando su identidad y controlando su acceso, por ejemplo Telnet y SSH.
- ✓ **De Ficheros.** Los servicios de ficheros consisten en ofrecer a la red grandes capacidades de almacenamiento para descongestionar o eliminar los discos de las estaciones de trabajo, permitiendo tanto el almacenamiento como la transferencia de ficheros, por ejemplo FTP.
- ✓ **Impresión.** Permite compartir de forma remota impresoras de alta calidad, capacidad y coste entre múltiples usuarios, reduciendo así el gasto.
- ✓ **Información.** Los servicios de información pueden servir ficheros en función de sus contenidos, como pueden ser los documentos de hipertexto, por ejemplo HTTP, o bien, pueden servir información para ser procesada por las aplicaciones, como es el caso de los servidores de bases de datos.
- ✓ **Comunicación.** Permiten la comunicación entre los usuarios a través de mensajes escritos, por ejemplo email o correo electrónico mediante el protocolo SMTP.

A veces, **un servicio toma como nombre, el nombre del protocolo del nivel de aplicación en el que está basado.** Por ejemplo, hablamos de servicio FTP, por basarse en el protocolo FTP.

En esta unidad, verás ejemplos de cómo programar en Java algunos de estos servicios, pero antes vamos a ver qué son los protocolos del nivel de aplicación.

Para saber más

Consulta el siguiente enlace si quieres profundizar en los servicios de red.

[Información sobre servicios de red.](#)

2.- Protocolos de comunicaciones del nivel de aplicación.

Caso práctico

Ana está entusiasmada con el nuevo reto y no quiere perder detalle. Le dice a Juan: "entonces.... ¿seguiremos utilizando sockets para los servicios que vamos añadir a la aplicación? Juan le responde -Los sockets siempre son una opción para programar servicios y clientes, pero para los que están basados en protocolos estándar, podemos encontrar algunas API que facilitan la tarea. Lo primero que haremos es ver algunos de los protocolos estándar del nivel de aplicación, porque quizá alguno de ellos nos de la respuesta a lo que buscamos.

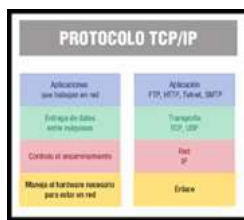


En la actualidad, el conjunto de protocolos **TCP/IP** constituyen el modelo más importante sobre el que se basa la comunicación de aplicaciones en red. Esto es debido, no sólo al espectacular desarrollo que ha tenido Internet, en los últimos años (recuerda que TCP/IP es el protocolo que utiliza Internet), sino porque también, TCP/IP ha ido cobrando cada vez más protagonismo en las redes locales y corporativas.

Dentro de la jerarquía de protocolos TCP/IP **la capa de Aplicación ocupa el nivel superior y es precisamente la que incluye los protocolos de alto nivel relacionados con los servicios en red** que te hemos indicado antes.

La **capa de Aplicación define los protocolos (normas y reglas) que utilizan las aplicaciones para intercambiar datos**. En realidad, hay tantos protocolos de nivel de aplicación como aplicaciones distintas; y además, continuamente se desarrollan nuevas aplicaciones, por lo que el número de protocolos y servicios sigue creciendo.

En la siguiente imagen puedes ver un esquema de la familia de protocolos TCP/IP y su organización en capas o niveles.



Pila de protocolos TCP/IP.

A continuación, vamos a destacar, por su importancia y gran uso, **algunos de los protocolos estándar del nivel de aplicación**:

- ✔ **FTP**. Protocolo para la transferencia de ficheros.
- ✔ **Telnet**. Protocolo que permite acceder a máquinas remotas a través de una red. Permite manejar por completo la computadora remota mediante un intérprete de comandos.
- ✔ **SMTP**. Protocolo que permite transferir correo electrónico. Recurre al protocolo de oficina postal **POP** para almacenar mensajes en los servidores, en sus versiones POP2 (dependiente de SMTP para el envío de mensajes) y POP3 (independiente de SMTP).
- ✔ **HTTP**. Protocolo de transferencia de hipertexto.
- ✔ **SSH**. Protocolo que permite gestionar remotamente a otra computadora de la red de forma segura.
- ✔ **NNTP**. Protocolo de Transferencia de Noticias (en inglés Network News Transfer Protocol).
- ✔ **IRC**. Chat Basado en Internet (en inglés Internet Relay Chat)
- ✔ **DNS**. Protocolo para traducir direcciones de red.

Autoevaluación

Señala si la afirmación siguiente es verdadera o falsa:

Ejemplos de protocolos del nivel de aplicación son: FTP, TCP, SMTP, HTTP.

☐ Verdadero ☐ Falso

Falso

La afirmación es falsa, TCP es un protocolo de nivel de transporte.

2.1.- Comunicación entre aplicaciones.

TCP/IP funciona sobre el concepto del modelo cliente/servidor, donde, como recordarás de unidades anteriores:

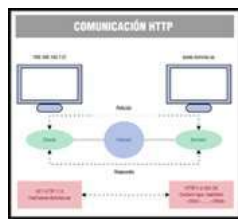
- ✓ **El Cliente** es el programa que ejecuta el usuario y solicita un servicio al servidor. Es quien inicia la comunicación.
- ✓ **El Servidor** es el programa que se ejecuta en una máquina (o varias) de red y ofrece un servicio (FTP, web, SMTP, etc.) a uno o múltiples clientes. Este proceso permanece a la espera de peticiones, las acepta y como respuesta, proporciona el servicio solicitado.

La capa de aplicación es el nivel que utilizan los programas para comunicarse a través de una red con otros programas. Los procesos que aparecen en esta capa, son aplicaciones específicas que pasan los datos al nivel de aplicación en el formato que internamente use el programa, y es codificado de acuerdo con un protocolo estándar. Una vez que los datos de la aplicación han sido codificados, en un protocolo estándar del nivel de aplicación, se pasan hacia abajo al siguiente nivel de la pila de protocolos TCP/IP.

En el nivel de transporte, las aplicaciones normalmente hacen uso de TCP y UDP, y son habitualmente asociados a un número de puerto. Por ejemplo, el servicio web mediante HTTP utiliza por defecto el puerto 80, el servicio FTP el puerto 21, etc.

Por ejemplo, la World Wide Web o Web, que usa el protocolo HTTP, sigue fielmente el modelo cliente/servidor:

- ✓ Los clientes son las aplicaciones que permiten consultar las páginas web (navegadores) y los servidores, las aplicaciones que suministran (sirven) páginas web.
- ✓ Cuando un usuario introduce una dirección web en el navegador (una URL), por ejemplo `http://www.dominio.es`, éste solicita mediante el protocolo HTTP la página web al servidor web que se ejecuta en la máquina donde está la página.
- ✓ El servidor web envía la página por Internet al cliente (navegador) que la solicitó, y éste último la muestra al usuario.



Diálogo entre cliente y servidor HTTP.

Los dos extremos dialogan siguiendo un protocolo de aplicación, tal y como puedes ver en la imagen que representa el ejemplo anterior.

- ✓ El cliente solicita el recurso web `www.dominio.es` mediante `GET HTTP/1.0`
- ✓ El servidor le contesta `HTTP/1.0 200 OK` (todo correcto) y le envía la página `html` solicitada.

Más adelante veremos varios ejemplos de cómo implementar un servicio web básico.

Para saber más

En el siguiente enlace dispones de un relación detallada de números de puerto y servicios que por defecto se le asocian, así como el protocolo de nivel de transporte que utilizan.

[Servicios y puertos asociados.](#)

Los [RFC](#) (Request For Comments) documentan los protocolos de aplicación estándar. En el siguiente enlace puedes consultar los RFC.

[Documentación RFC.](#)

2.2.- Conexión, transmisión y desconexión.

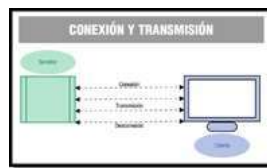
Los protocolos de aplicación se comunican con el nivel de transporte mediante un API, denominada **API Socket**, y que en el caso de Java viene implementada mediante las clases del paquete `java.net` como `Socket` y `ServerSocket`.

Como recordarás, un socket Java es la representación de una conexión para la transmisión de información entre dos ordenadores distintos o entre un ordenador y él mismo. Esta abstracción de medio nivel, permite despreocuparse de lo que está pasando en capas inferiores.

Dentro de una red, un socket es único pues viene caracterizado por cinco parámetros: el protocolo usado (HTTP, FTP, etc.), dos direcciones IP (la del equipo local o donde reside el proceso cliente, y la del equipo remoto o donde reside el proceso servidor) y dos puertos (uno local y otro remoto)

Te recordamos los pasos que se siguen para establecer, mantener y cerrar una conexión TCP/IP:

- ✔ Se crean los sockets en el cliente y el servidor
- ✔ El servidor establece el puerto por el que proporciona el servicio
- ✔ El servidor permanece a la escucha de las peticiones de los clientes.
- ✔ Un cliente conecta con el servidor.
- ✔ El servidor acepta la conexión.
- ✔ Se realiza el intercambio de datos.
- ✔ El cliente o el servidor, o ambos, cierran la conexión.



Esquema conexión-transmisión-desconexión cliente/servidor.

Autoevaluación

Señala si la afirmación siguiente es verdadera o falsa:

Los procesos de aplicación se comunican con la capa de transporte mediante un API.

☐ Verdadero ☐ Falso

Verdadero

La afirmación es verdadera, este API es el API socket.

2.3.- DNS y resolución de nombres.

Todas las computadoras y dispositivos conectados a una red TCP/IP se identifican mediante una dirección IP, como por ejemplo 117.142.234.125.

En su forma actual (IPv4), una dirección IP se compone de cuatro bytes sin signo (de 0 a 254) separados por puntos para que resulte más legible, tal y como has visto en el ejemplo anterior. Por supuesto, se trata de valores ideales para los ordenadores, pero para los seres humanos que quieran acordarse de la IP de un nodo en concreto de la red, son todo un problema de memoria.

El sistema DNS o Sistema de Nombres de Dominio es el mecanismo que se inventó, para que los nodos que ofrecen algún tipo de servicio interesante tengan un nombre fácil de recordar, lo que se denomina un nombre de dominio, como por ejemplo **www.todofp.es**.



DNS es un sistema de nomenclatura jerárquica para computadoras, servicios o cualquier recurso conectado a Internet o a una red privada.

El objetivo principal de los nombres de dominio y del servicio DNS, es traducir o resolver nombres (por ejemplo **www.dominio.es**) en las direcciones IP (identificador binario) de cada equipo conectado a la red, con el propósito de poder localizar y direccionar estos equipos dentro de la red.

Sin la ayuda del servicio DNS, los usuarios de una red TCP/IP tendrían que acceder a cada servicio de la misma utilizando la dirección IP del nodo.

Además **el servicio DNS proporciona otras ventajas:**

- ✓ Permite que una misma dirección IP sea compartida por varios dominios.
- ✓ Permite que un mismo dominio se corresponda con diferentes direcciones IP.
- ✓ Permite que cualquier servicio de red pueda cambiar de nodo, y por tanto de IP, sin cambiar de nombre de dominio.

Autoevaluación

Señala si la afirmación siguiente es verdadera o falsa:

El servicio DNS permite resolver una IP en su único nombre de dominio.

☐ Verdadero ☐ Falso

Falso

La afirmación es falsa, pues una misma IP puede ser compartida por más de un dominio.

Para saber más

Puedes profundizar en el funcionamiento del servicio DNS en el siguiente enlace:

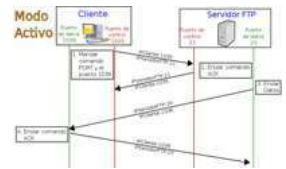
[Información detallada sobre el servicio DNS.](#)

2.4.- El protocolo FTP.

El protocolo FTP o Protocolo de Transferencia de Archivos proporciona un mecanismo estándar de transferencia de archivos entre sistemas a través de redes TCP/IP.

Las **principales prestaciones de un servicio basado en el protocolo FTP o servicio FTP** son las siguientes:

- ✓ Permite el intercambio de archivos entre máquinas remotas a través de la red.
- ✓ Consigue una conexión y una transferencia de datos muy rápidas.



Sin embargo, **el servicio FTP adolece de una importante deficiencia en cuanto a seguridad:**

- ✓ La información y contraseñas se transmiten en texto plano. Esto está diseñado así, para conseguir una mayor velocidad de transferencia. Al realizar la transmisión en texto plano, un posible atacante puede capturar este tráfico, acceder al servidor y/o apropiarse de los archivos transferidos.

Este problema de seguridad, se puede solucionar mediante la encriptación de todo el tráfico de información, a través del protocolo no estándar SFTP usando SSH o del protocolo FTPS usando SSL. De encriptación ya hablaremos más adelante, en otra unidad de este módulo.

¿Cómo funciona el servicio FTP? Es un servicio basado en una arquitectura cliente/servidor y, como tal, seguirá las pautas generales de funcionamiento de este modelo, en ese caso:

- ✓ El servidor proporciona su servicio a través de dos puertos:
 - El puerto 20, para transferencia de datos.
 - El puerto 21, para transferencia de órdenes de control, como conexión y desconexión.
- ✓ El cliente se conecta al servidor haciendo uso de un puerto local mayor de 1024 y tomando como puerto destino del servidor el 21.

Las **principales características del servicio FTP** son las siguientes:

- ✓ La **conexión de un usuario remoto** al servidor FTP puede hacerse como: usuario del sistema (debe tener una cuenta de acceso), usuario genérico (utiliza la cuenta anonymous, esto es, usuario anónimo), usuario virtual (no requiere una cuenta local del sistema).
- ✓ El **acceso al sistema de archivos** es más o menos limitado, según el tipo de usuario que se conecte y sus privilegios. Por ejemplo, el usuario anónimo solo tendrá acceso al directorio público que establece el administrador por defecto.
- ✓ El servicio FTP soporta dos **modos de conexión**: modo activo y modo pasivo.
 - **Modo activo.** Es la forma nativa de funcionamiento (esquema ampliable de la derecha).
 - Se establecen **dos conexiones distintas**: la petición de transferencia por parte del cliente y la atención a dicha petición, iniciada por el servidor. De manera que, si el cliente está protegido con un cortafuegos, deberá configurarlo para que permita esta petición de conexión entrante a través de un puerto que, normalmente, está cerrado por motivos de seguridad.
 - **Modo pasivo.** El cliente sigue iniciando la conexión, pero el problema del cortafuegos se traslada al servidor.

Autoevaluación

Señala si la afirmación siguiente es verdadera o falsa:

FTP es un servicio que utiliza el puerto 21 para transmitir datos y el 20 para transmitir órdenes de control.

☐ Verdadero ☐ Falso

Falso

La afirmación es falsa, es justo lo contrario.

Para saber más

Te recomendamos el siguiente enlace para seguir profundizando en el protocolo FTP.

[Protocolo FTP.](#)

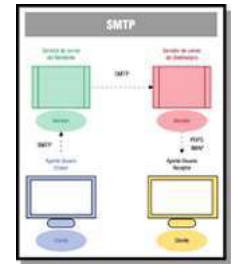
2.5.- Los protocolos SMTP y POP3.

El correo electrónico es un servicio que permite a los usuarios enviar y recibir mensajes y archivos rápidamente a través de la red.

- ✓ Principalmente se usa este nombre para denominar al sistema que provee este servicio en Internet, mediante el protocolo SMTP o Protocolo Simple de Transferencia de Correo, aunque por extensión también puede verse aplicado a sistemas análogos que usen otras tecnologías.
- ✓ Por medio de mensajes de correo electrónico se puede enviar, no solamente texto, sino todo tipo de documentos digitales.

El servicio de correo basado en el protocolo SMTP sigue el modelo cliente/servidor, por lo que el trabajo se distribuye entre el programa Servidor y el Cliente. Te indicamos a continuación, **algunas consideraciones importantes sobre el servicio de correo a través de SMTP**:

- ✓ El servidor mantiene las cuentas de los usuarios así como los buzones correspondientes.
- ✓ Los clientes de correo gestionan la descarga de mensajes así como su elaboración.
- ✓ El servicio SMTP utiliza el puerto 25.
- ✓ El protocolo SMTP se encarga del transporte del correo saliente desde la máquina del usuario remitente hasta el servidor que almacene los mensajes de los destinatarios.
 - El usuario remitente redacta su mensaje y lo envía hacia su servidor de correo.
 - Desde allí se reenvía al servidor del destinatario, quién lo descarga del buzón en la máquina local mediante el protocolo POP3, o la consulta vía web, haciendo uso del protocolo IMAP.



Esquema protocolos SMTP.

¿Cómo funciona SMTP y qué mensajes intercambian cliente y servidor? En el siguiente enlace dispones de una explicación del funcionamiento del protocolo SMTP en líneas generales (Resumen simple de funcionamiento):

[Funcionamiento del protocolo SMTP.](#)

Para saber más

Si quieres profundizar sobre el protocolo POP3, consulta el siguiente enlace:

[Protocolo POP3.](#)

En este otro enlace descubrirás las ventajas de IMAP sobre POP3:

[Protocolo IMAP.](#)

2.6.- El protocolo HTTP.

El protocolo HTTP o Protocolo de Transferencia de Hipertexto es un conjunto de normas que posibilitan la comunicación entre servidor y cliente, permitiendo la transmisión de información entre ambos. La información transferida son las conocidas páginas HTML o páginas web. Se trata del método más común de intercambio de información en la World Wide Web.

HTTP define tanto la sintaxis como la semántica que utilizan clientes y servidores para comunicarse. Algunas **consideraciones importantes sobre HTTP** son las siguientes:

- ✓ Es un protocolo que sigue el **esquema petición-respuesta** entre un cliente y un servidor.
- ✓ Utiliza por defecto el **puerto 80**
- ✓ Al **cliente que efectúa la petición**, por ejemplo un navegador web, se le conoce como agente del usuario (user agent).
- ✓ A la **información transmitida se la llama recurso y se la identifica mediante un localizador uniforme de recursos (URL)**.
Por ejemplo: `http://www.iesalandalus.org/organizacion.htm`
- ✓ **Los recursos pueden ser** archivos, el resultado de la ejecución de un programa, una consulta a una base de datos, la traducción automática de un documento, etc.



El **funcionamiento esquemático del protocolo HTTP** es el siguiente:

- ✓ El usuario especifica en el cliente web la dirección del recurso a localizar con el siguiente formato: `http://dirección[:puerto][ruta]`, por ejemplo: `http://www.iesalandalus.org/organizacion.htm`
- ✓ El cliente web descompone la información de la URL diferenciando el protocolo de acceso, la IP o nombre de dominio del servidor, el puerto y otros parámetros si los hubiera.
- ✓ El cliente web establece una conexión al servidor y solicita el recurso web mediante un mensaje al servidor, encabezado por un método, por ejemplo `GET /organizacion.htm HTTP/1.1` y otras líneas.
- ✓ El servidor contesta con un mensaje encabezado con la línea `HTTP/1.1 200 OK`, si existe la página y la envía, o bien envía un código de error.
- ✓ El cliente web interpreta el código HTML recibido.
- ✓ Se cierra la conexión.

En el siguiente enlace dispones de un ejemplo de diálogo entre cliente y servidor utilizando el protocolo HTTP. Más adelante, estudiaremos algunos detalles más de este protocolo para poder programar un servicio web básico.

[Ejemplo de diálogo entre cliente y servidor usando el protocolo HTTP.](#)

HTTP es un protocolo sin estado, lo que significa que no recuerda nada relativo a conexiones anteriores a la actual. Algunas aplicaciones necesitan que se guarde el estado y para ello hacen uso de lo que se conoce como cookie.

Autoevaluación

Señala si la afirmación siguiente es verdadera o falsa:

HTTP es un protocolo al igual que URL.

☐ Verdadero ☐ Falso

Falso

La afirmación es falsa, http es un protocolo, pero URL no es un protocolo, es la dirección de un recurso en la web.

Para saber más

¿No sabes que son y para que sirven las cookies? Consulta este enlace para aprender sobre ellas.

[Información sobre cookies.](#)

Si quieres profundizar en el funcionamiento del protocolo HTTP te recomendamos que visites este enlace:

[Información sobre protocolo HTTP.](#)

3.- Bibliotecas de clases y componentes Java.

Caso práctico

Ana le dice a **Juan**, -ahora tenemos que ver qué clases nos proporciona Java para implementar servicios y clientes en red ¿Habrás alguna otra además de `java.net`?

Juan le contesta, -la biblioteca `java.net` es muy extensa y es la que proporciona prácticamente todas las funcionalidades de programación en red. Vamos a estudiarla a fondo y ver qué clases nos pueden servir para nuestra aplicación.

Ana dice, -me parece perfecto. ¿Empezamos?



Java se ha construido con extensas capacidades de interconexión TCP/IP y soporta diferentes niveles de conectividad en red, facilitando la creación de aplicaciones cliente/servidor y generación de servicios en red. Así por ejemplo: permite abrir una URL como por ejemplo `http://www.dominio.es/public/archivo.pdf`, permite realizar una invocación de métodos remotos, RMI, o permite trabajar con sockets.

El paquete principal que proporciona el API de Java para programar aplicaciones con comunicaciones en red es:

- ✔ `java.net`. Este paquete soporta clases para generar diferentes servicios de red, servidores y clientes.



Otros paquetes Java para comunicaciones y servicios en red son::

- ✔ `java.rmi`. Paquete que permite implementar una interface de control remoto (RMI).
- ✔ `javax.mail`. Permite implementar sistemas de correo electrónico.

Para ciertos servicios estándar, Java no proporciona objetos predefinidos y por tanto una solución fácil para generarlos es recurrir a bibliotecas externas, como por ejemplo, el API proporcionado por Apache Commons Net (las bibliotecas `org.apache.commons.net`). Esta API permite implementar aplicaciones cliente para los protocolos estándar más populares, como: Telnet, FTP, o FTP sobre HTTP entre otros.

Para saber más

En el siguiente enlace puedes consultar todas las funcionalidades que ofrece el API Apache Commons Net.

[Bibliotecas de Apache Commons Net.](#)

3.1.- Objetos predefinidos.

El paquete `java.net`, proporciona una API que se puede dividir en dos niveles:

- ✓ **Una API de bajo nivel**, que permite representar los siguientes objetos:
 - **Direcciones.** Son los identificadores de red, esto es, las direcciones IP.
 - Clase `InetAddress`. Implementa una dirección IP.
 - **Sockets.** Son los mecanismos básicos de comunicación bidireccional de datos.
 - Clase `Socket`. Implementa un extremo de una conexión bidireccional.
 - Clase `ServerSocket`. Implementa un `socket` que los servidores pueden utilizar para escuchar y aceptar peticiones de clientes.
 - Clase `DatagramSocket`. Implementa un `socket` para el envío y recepción de datagramas.
 - Clase `MulticastSocket`. Representa un `socket` datagrama, útil para enviar paquetes multidifusión.
 - **Interfaces.** Describen las interfaces de red.
 - Clase `NetworkInterface`. Representa una interfaz de red compuesta por un nombre y una lista de direcciones IP asignadas a esta interfaz.
- ✓ **Una API de alto nivel**, que se ocupa de representar los siguientes objetos, mediante las clases que te indicamos:
 - **URI.** Representan los identificadores de recursos universales.
 - Clase `URI`.
 - **URL.** Representan localizadores de recursos universales.
 - Clase `URL`. Representa una dirección URL.
 - **Conexiones.** Representa las conexiones con el recurso apuntado por URL.
 - Clase `URLConnection`. Es la superclase de todas las clases que representan un enlace de comunicaciones entre la aplicación y una URL.
 - Clase `HttpURLConnection`. Representa una `URLConnection` con soporte para HTTP y con ciertas características especiales.



Mediante las clases de alto nivel se consigue una mayor abstracción, de manera que, esto facilita bastante la creación de programas que acceden a los recursos de la red. A continuación veremos algunos ejemplos.

Autoevaluación

Señala si la afirmación siguiente es verdadera o falsa:

La clase `URLConnection` representa una enlace entre entre un recurso URL y la aplicación.

☐ Verdadero ☐ Falso

Verdadero

La afirmación es verdadera. Es la superclase de todas las clases que representan un enlace de comunicaciones entre la aplicación y la URL.

Para saber más

En el siguiente enlace puedes consultar el paquete `java.net` en toda su extensión.

[Paquete java.net.](#)

3.2.- Métodos y ejemplos de uso de InetAddress.

La clase `InetAddress` proporciona objetos que puedes utilizar para manipular tanto direcciones IP como nombres de dominio. También proporciona métodos para resolver los nombres de host a sus direcciones IP y viceversa.

Una instancia de `InetAddress` consta de una dirección IP y en algunos casos también del nombre de host asociado. Esto último depende de si se ha creado con el nombre de host o bien ya se ha aplicado la resolución de nombres.

Esta clase no tiene constructores. Sin embargo, `InetAddress` **dispone de métodos estáticos que devuelven objetos `InetAddress`**. Te indicamos cuáles son esos métodos:

- ✓ `getLocalHost()`. Devuelve un objeto de tipo `InetAddress` con los datos de direccionamiento de mi equipo en la red local (no del conocido `localhost`).
- ✓ `getByName(String host)`. Devuelve un objeto de tipo `InetAddress` con los datos de direccionamiento del host que le pasamos como parámetro. Donde el parámetro `host` tiene que ser un nombre o IP válido, ya sea de Internet (como `iesalandalus.org` o `195.78.228.161`), o de tu red de área local (como `documentos.servidor` o `192.168.0.5`). Incluso puedes poner `localhost`, o cualquier otra IP o nombre NetBIOS de tu red local.
- ✓ `getAllByName(String host)`. Devuelve un array de objetos de tipo `InetAddress` con los datos de direccionamiento del host pasado como parámetro. Recuerda que en Internet es frecuente que un mismo dominio tenga a su disposición más de una IP.

Todos estos métodos pueden generar una excepción `UnknownHostException` si no pueden resolver el nombre pasado como parámetro.

Algunos **métodos interesantes de un objeto `InetAddress`** para resolver nombres son los siguientes:

- ✓ `getHostAddress()`. Devuelve en una cadena de texto la correspondiente IP.
- ✓ `getAddress()`. Devuelve un array formado por los grupos de bytes de la IP correspondiente.

Otros métodos interesantes de esta clase son:

- ✓ `getHostName()`. Devuelve en una cadena de texto el nombre del host.
- ✓ `isReachable(int tiempo)`. Devuelve TRUE o FALSE dependiendo de si la dirección es alcanzable en el tiempo indicado en el parámetro.

En el siguiente enlace tienes un ejemplo de uso de `InetAddress`. Con este ejemplo tratamos de ilustrar el uso de los métodos anteriores para resolver algunos nombres a su dirección o direcciones IP, tanto en la red local como en Internet. Por ello, para probarlo, debes tener conexión a Internet, en otro caso sólo se ejecutará la parte relativa a la red local, y se lanzará la correspondiente excepción al intentar resolver nombres de Internet.

[Ejemplo de uso de la clase `InetAddress`](#) (0.01 MB)

Autoevaluación

Señala si la afirmación siguiente es verdadera o falsa:

El método `getLocalHost()` devuelve la IP del equipo de nombre `localhost`.

☐ Verdadero ☐ Falso

Falso

La afirmación es falsa, devuelve un objeto `InetAddress` asociado a mi equipo en la red local.

Citas para pensar

"Es absolutamente imposible demostrarlo todo."

Aristóteles



3.3.- Programación con URL.

La programación de **URL** se produce a un nivel más alto que la programación de **sockets** y ésto, puede facilitar la creación de aplicaciones que acceden a recursos de la red.

¿Tienes claro lo que es una URL y las partes en las que se puede descomponer? Vamos a verlo.

Una URL, Localizador Uniforme de Recursos, representa una dirección a un recurso de la World Wide Web. Un **recurso** puede ser algo tan simple como un archivo o un directorio, o puede ser una referencia a un objeto más complicado, como una consulta a una base de datos, el resultado de la ejecución de un programa, etc.

Consideraciones sobre una URL:

- ✓ Puede referirse a sitios web, ficheros, sitios ftp, direcciones de correo electrónico, etc.
- ✓ La **estructura de una URL** se puede dividir en varias partes, tal y como puede ver en la imagen:



- ✓ **Protocolo.** El protocolo que se usa para comunicar.
- ✓ **Nombre de host.** Nombre del host que proporciona el servicio o servidor.
- ✓ **Puerto.** El puerto de red en el servidor para conectarse. Si no se especifica, se utiliza el puerto por defecto para el protocolo.
- ✓ **Ruta.** Es la ruta o path al recurso en el servidor.
- ✓ **Referencia.** Es un fragmento que indica una parte específica dentro del recurso especificado.

Por ejemplo, algunas posibles direcciones URL serían las siguientes:

- ✓ `http://www.iesalandalus.org/organizacion.htm`, el protocolo utilizado es el http, el nombre del host `www.iesalandalus.org` y la ruta es `organizacion.htm`, que en este caso es una página html. Al no indicar puerto, se toma el puerto por defecto para HTTP que es el 80.
- ✓ `http://www.iesalandalus.org:85/organizacion.htm`, en este caso se está indicando como puerto el 85.
- ✓ `http://www.dominio.es/public/pag.html#apartado1`, en este caso se especifica como ruta `/public/pag.html#apartado1`, por lo que una vez recuperado el archivo `pag.htm` se está indicando mediante la referencia `#apartado1` que interesa el apartado1 dentro de esa página.

Debes conocer

En una URL se pueden especificar otros protocolos además de HTTP. Consulta el siguiente enlace para ver esos protocolos y más ejemplos de posibles URL.

[Partes de una URL y posibles protocolos.](#)

La clase **URL** de Java permite representar un URL.

Utilizando la clase **URL**, se puede establecer una conexión con cualquier recurso que se encuentre en Internet, o en nuestra Intranet. Una vez establecida la conexión, invocando los métodos apropiados sobre ese objeto **URL** se puede obtener el contenido del recurso en el cliente. Esto es una idea potentísima, pues permite, en teoría, descargar el contenido de cualquier recurso en el cliente, incluso aunque se requiriera un protocolo que no existía cuando se escribió el programa.

Pero la clase **URL** también proporciona una forma alternativa de conectar un ordenador con otro y compartir datos, basándose en **streams**. Esto último, es algo que también se puede hacer con la programación de **sockets**, tal y como has visto en las unidades anteriores.

Autoevaluación

Señala si la afirmación siguiente es verdadera o falsa:

En una URL el protocolo que se debe especificar es HTTP.

☐ Verdadero ☐ Falso

Falso

La afirmación es falsa, pues se admiten otros protocolos como por ejemplo ftp.

3.4.- Crear y analizar objetos URL.



¿Cómo podemos crear objetos URL? La clase URL dispone de diversos constructores para crear objetos tipo URL que se diferencian en la forma de pasarle la dirección URL. Por ejemplo, se puede crear un objeto URL:

- ✓ A partir de todos o algunos de los elementos que forman parte de una URL, como por ejemplo:
`URL url=new URL("http", "www.iesalandalus.org",80,"index.htm")`. Crea un objeto URL a partir de los componentes indicados (protocolo, host, puerto, fichero), esto es, crea la URL:
`http://www.iesalandalus.org:80/index.htm`.
- ✓ A partir de la cadena especificada, dejando que el sistema utilice todos los valores por defecto que tiene definidos, como por ejemplo:
`URL url=new URL("http://www.iesalandalus.org")` que crearía la URL : `http://www.iesalandalus.org`.
- ✓ A partir de una ruta relativa pasada como primer parámetro.
- ✓ A partir de las especificaciones indicadas y un manejador del protocolo que conoce cómo comunicarse con el servidor.

Cada uno de los constructores de URL puede lanzar una `MalformedURLException` si los argumentos del constructor son nulos o el protocolo es desconocido. Lo normal, es capturar y manejar esta excepción mediante un bloque `try- catch`.

En el siguiente enlace dispones de ejemplos que ilustran diferentes formas de crear un objeto URL.

[La clase URL y ejemplos.](#)

Las URLs son objetos de una sola escritura. Lo que significa, que una vez que has creado un objeto URL no se puede cambiar ninguno de sus atributos (protocolo, nombre del host, nombre del fichero ni número de puerto).

Se puede analizar y descomponer una URL utilizando los siguientes métodos:

- ✓ `getProtocol()`. Obtiene el protocolo de la URL.
- ✓ `getHost()`. Obtiene el host de la URL.
- ✓ `getPort()`. Obtiene el puerto de la URL, si no se ha especificado obtiene -1.
- ✓ `getDefaultPort()`. Obtiene el puerto por defecto asociado a la URL, si no lo tiene obtiene -1.
- ✓ `getFile()`. Obtiene el fichero de la URL o una cadena vacía si no existe.
- ✓ `getRef()`. Obtiene la referencia de la URL o `null` si no la tiene.

En el siguiente enlace puedes ver un ejemplo de cómo analizar o descomponer una URL.

[Analizar una URL.](#)

Autoevaluación

Señala las opciones correctas. Métodos que permiten analizar una URL son:

- ☐ El método `getURL()` que obtiene cada parte de la URL.

- ☐ El método `getHost()` que obtiene el host.

- ☐ El método `URL("direccion")` que la analiza de forma completa.

- ☐ El método `getProtocol()` que obtiene el protocolo.

Mostrar retroalimentación

Solución

1. Incorrecto
2. Correcto
3. Incorrecto
4. Correcto



3.5.- Leer y escribir a través de una URLConnection.

Un **objeto** `URLConnection` se puede utilizar para leer desde y escribir hacia el recurso al que hace referencia el objeto `URL`.

De entre los muchos **métodos** que nos permiten trabajar con conexiones `URL` vamos a centrarnos en primer lugar en los siguientes:

- `URL.openConnection()`. Devuelve un objeto `URLConnection` que representa una nueva conexión con el recurso remoto al que se refiere la `URL`.
- `URL.openStream()`. Abre una conexión a esta dirección `URL` y devuelve un `InputStream` para la lectura de esa conexión. Es una abreviatura de: `openConnection().getInputStream()`.

Te mostramos a continuación dos ejemplos muy sencillos que leen una `URL`, basándose tan solo en estos dos métodos. Los pasos a seguir para leer la `URL` son:

- Crear el objeto `URL` mediante `URL url=new URL(...);`
- Obtener una conexión con el recurso especificado mediante `URL.openConnection()`.
- Abrir conexión con esa `URL` mediante `URL.openStream()`.
- Manejar los flujos necesarios para realizar la lectura

Por ejemplo, podemos utilizar objetos `URL` para leer un archivo de texto y almacenarlo en un fichero local, tal y como puedes ver en el siguiente segmento de código:

```
URL url = new URL("http://www.javatips.net/");
URLConnection urlCon = url.openConnection();
urlCon.setConnectTimeout(10000);
urlCon.setReadTimeout(10000);
InputStream inputStream = urlCon.getInputStream();
FileOutputStream outputStream = new FileOutputStream("url.txt");
byte[] buffer = new byte[1024];
int bytesRead;
while ((bytesRead = inputStream.read(buffer)) != -1) {
    outputStream.write(buffer, 0, bytesRead);
}
```

[Código Java ejemplo de lectura de archivo mediante URL.](#) (0,01 MB)

También podemos utilizar objetos `URL` para leer una `URL` y analizar su código fuente, tal y como hacen los buscadores, optimizadores de código o validadores de código. En el siguiente enlace dispones de un ejemplo de este uso, desde donde puedes descargar el código y probarlo en tu equipo.

[Ejemplo java para leer una URL.](#)

En el siguiente enlace dispones de más ejemplos para leer y escribir mediante un objeto `URLConnection`.

[Leer y escribir desde una conexión URL.](#)

Autoevaluación

Señala si la afirmación siguiente es verdadera o falsa:

El método `URL.openStream()` devuelve un objeto `URLConnection`.

☐ Verdadero ☐ Falso

Falso

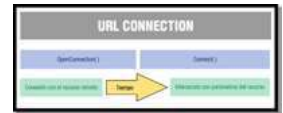
La afirmación es falsa, devuelve un objeto `InputStream` para la lectura de la `URL`.



3.6.- Trabajar con el contenido de una URL.

El método que permite **obtener el contenido de un objeto URL** es:

- ✔ **URL.getContent()**. Devuelve el contenido de esa URL. Este método determina en primer lugar el tipo de contenido del objeto llamando al método **getContentType()**. Si esa es la primera vez que la aplicación ha visto ese tipo de contenido específico, se crea un manejador para dicho tipo de contenido.



Mediante este método y otros proporcionados por la clase **URLConnection**, veremos al final de este apartado, un ejemplo de cómo examinar el contenido del objeto URL y realizar la tarea que interese.

Crear una conexión URL realmente implica los siguientes pasos:

- ✔ Crear un objeto **URLConnection**, éste se crea cuando se llama al método **openConnection()** de un objeto **URL**.
- ✔ Establecer los parámetros y propiedades de la petición.
- ✔ Establecer la conexión utilizando el método **connect()**.
- ✔ Se puede obtener información sobre la cabecera o/y obtener el recurso remoto.

A partir de un objeto **URLConnection**, se puede obtener información muy útil sobre la conexión que se haya establecido y existen muchos métodos en la clase **URLConnection** que se pueden utilizar para examinar y manipular el objeto que se crea de este tipo.

Vamos a ver algunos métodos que serán de utilidad en nuestro ejemplo:

- ✔ **connect()**. Establece una conexión entre la aplicación (el cliente) y el recurso (el servidor), permite interactuar con el recurso y consultar los tipos de cabeceras y contenidos para determinar el tipo de recurso de que se trata.
- ✔ **getContentType()**. Devuelve el valor del campo de cabecera **content-type** o **null** si no está definido.
- ✔ **getContentLength()**. Devuelve el valor del campo de cabecera **content-length** o **-1** si no está definido.
- ✔ **getLastModified()**. Devuelve la fecha de la última modificación del recurso.

A continuación dispones de un documento que muestra la creación de una aplicación para acceder a recursos de Internet a través de objetos **URL**, esto es, un cliente web.

[Acceso a recursos. URL y URLConnection.](#)

[Resumen textual alternativo](#)

Desde el siguiente enlace te puedes descargar el proyecto completo.

[Acceso a recursos con URL y URLConnection.](#) (0,09 MB)

Para saber más

Se puede utilizar la clase **URL** para escribir manejadores de protocolos o cosas parecidas. Para ello, será necesario conocer en profundidad la clase **URLConnection**. Consulta este enlace para ello.

[Clase URLConnection.](#)

Desde este enlace puedes consultar todos los parámetros de las cabeceras del protocolo HTTP.

[Cabeceras del protocolo HTTP.](#)

4.- Programación de aplicaciones cliente.

Caso práctico

María ha ido hoy a ver a **Juan** y **Ana**, pues **Ada** le ha comentado que quizá le venga bien ver la marcha del proyecto. **María**, que lleva el mantenimiento de varias webs a la empresa, está interesada en conocer más de cerca los entresijos de determinadas aplicaciones de Internet. **Juan** y **Ana** la saludan y **Juan** le dice: -No has podido elegir mejor día para trabajar con nosotros, **María**. Justo hoy veremos algunas API que facilitan mucho la programación de ciertos clientes de Internet al trabajar a un nivel alto, por encima de los **sockets**.

María sonríe y dice: -Perfecto, para sorpresa vuestra os diré que conozco un API para programar clientes de correo electrónico. -**Ana** dice- ¿Te refieres a **javax.mail**?



La programación de aplicaciones cliente y aplicaciones servidor que vamos a realizar, está basada en el uso de los protocolos estándar de la capa de aplicación, por tanto, cuando programemos una aplicación servidor basada en el protocolo HTTP, por ejemplo, diremos también que se está programando un servicio o servidor HTTP, y si lo que programamos es el cliente, hablaremos de cliente HTTP.

Nos vamos a centrar en la programación de aplicaciones cliente para los protocolos HTTP, FTP, SMTP y TELNET. Su programación se realizará mediante bibliotecas especiales que proporcionan ciertos objetos que facilitan la tarea de programación, y en algunos casos veremos también cómo programar esas mismas aplicaciones cliente mediante **sockets**.



Es imprescindible un mínimo conocimiento de cómo funciona el protocolo sobre el que vamos a construir un cliente, para tener claro el intercambio de mensajes que realiza con el servidor. Por tanto, no repares en volver a los primeros apartados o consultar los enlaces para saber más, donde se explica algo sobre el funcionamiento de estos protocolos. Esto es más necesario, si la programación se realiza a niveles relativamente bajos, como por ejemplo cuando utilizamos **sockets**.

Citas para pensar

"Las ciencias tienen las raíces amargas, pero muy dulces los frutos."

Aristóteles

4.1.- Programación de un cliente HTTP.

Como ya te hemos comentado anteriormente, HTTP se basa en sencillas operaciones de solicitud/respuesta.

- ✔ Un cliente establece una conexión con un servidor y envía un mensaje con los datos de la solicitud.
- ✔ El servidor responde con un mensaje similar, que contiene el estado de la operación y su posible resultado.



¿Es un cliente HTTP el ejemplo que hemos visto sobre el acceso a recursos de la red mediante URL, utilizando las clases `URL` y `URLConnection`? Efectivamente, se trataba de un cliente web básico, en particular un cliente http muy básico, que no realiza por ejemplo la traducción de una página html tal y como lo hace un navegador.

Al programar aplicaciones con las clases `URL` y `URLConnection`, lo hacemos en un nivel alto, de manera que queda oculta toda la operatoria que era tan explícita al programar un cliente con `sockets`.

En el siguiente enlace puedes consultar otro ejemplo de cliente HTTP realizado con `sockets` y cuyo comportamiento es similar a los ejemplos vistos anteriormente mediante las clases `URLs`.

[Ejemplo de cliente HTTP mediante sockets.](#)

Para saber más

¿Sabías que puedes ver las cabeceras o encabezados que envía el servidor web a tu navegador y viceversa mediante una extensión del navegador Firefox? Mira el siguiente enlace y lo podrás comprobar.

[Ver las cabeceras que envían navegador y servidor web.](#)

4.2.- Bibliotecas para programar un cliente FTP.

Java no dispone de bibliotecas específicas para programar clientes y servidores de FTP. Pero afortunadamente, la organización de software libre The Apache software Foundation proporciona una API para implementar clientes FTP en tus aplicaciones.

El **paquete de la API de Apache para trabajar con FTP** es `org.apache.commons.net.ftp`. Este paquete proporciona, entre otras, las siguientes **clases**:



- ✔ **Clase FTP.** Proporciona las funcionalidades básicas para implementar un cliente FTP. Esta clase hereda de `SocketClient`.
- ✔ **Clase FTPReply.** Permite almacenar los valores devueltos por el servidor como código de respuesta a las peticiones del cliente.
- ✔ **Clase FTPClient.** Encapsula toda la funcionalidad que necesitamos para almacenar y recuperar archivos de un servidor FTP, encargándose de todos los detalles de bajo nivel para su interacción con el servidor. Esta clase hereda de `SocketClient`.
- ✔ **Clase FTPClientConfig.** Proporciona un forma alternativa de configurar objetos `FTPClient`.
- ✔ **Clase FTPSClient.** Proporciona FTP seguro, sobre SSL. Esta clase hereda de `FTPClient`.

En el siguiente recurso didáctico tienes los pasos a seguir para descargar la API de Apache y realizar su integración en NetBeans.

[API Apache. Descarga e integración NetBeans](#)

[Resumen textual alternativo](#)

Citas para pensar

"Lo que sabemos es una gota de agua; lo que ignoramos es el océano "

Isaac Newton

Para saber más

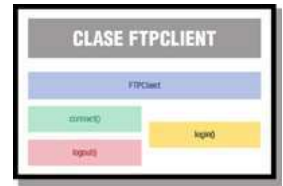
En el siguiente enlace dispones de toda la información sobre el API de Apache para el servicio FTP.

[API de Apache para el servicio FTP.](#)

4.3.- Programación de un cliente FTP.

Una forma sencilla de **crear un cliente FTP** es **mediante la clase FTPClient**.

Una vez creado el cliente, como en cualquier objeto `SocketClient` habrá que seguir el siguiente **esquema básico de trabajo**:



- ✓ **Realizar la conexión del cliente con el servidor.** El método `connect(InetAddress host)` realiza la conexión del cliente con el servidor de nombre `host`, abriendo un objeto `Socket` conectado al `host` remoto en el puerto por defecto.
- ✓ **Comprobar la conexión.** El método `getReplyCode()` devuelve un código de respuesta del servidor indicativo de el éxito o fracaso de la conexión.
- ✓ **Validar usuario.** El método `login(String usuario, String password)` permite esa validación.
- ✓ **Realizar operaciones contra el servidor.** Por ejemplo:
 - Listar todos los ficheros disponibles en una determinada carpeta remota mediante el método `listNames()`.
 - Recuperar el contenido de un fichero remoto mediante `retrieveFile(String rutaRemota, OutputStream ficheroLocal)` y transferirlo al equipo local para escribirlo en el `ficheroLocal` especificado.
- ✓ **Desconexión del servidor.** El método `disconnect()` o `logout()` realiza la desconexión del servidor.

Ten en cuenta, que durante todo este proceso puede generarse tanto una `SocketException` si se superó el tiempo de espera para conectar con el servidor, o una `IOException` si no se tiene acceso al fichero especificado.

En el siguiente enlace, tienes un ejemplo de programación de un cliente FTP, que se conecta a un servidor remoto y se descarga un fichero. Ten en cuenta que para poder ejecutarlo, puede que tengas que desactivar cualquier cortafuegos o firewall que tengas activo.

[Código del proyecto Java para programar un cliente FTP.](#) (0,01 MB)

Autoevaluación

Señala si la afirmación siguiente es verdadera o falsa:

El método `FTPClient.login (user, password)` realiza la conexión a un servidor FTP como el usuario indicado.

☐ Verdadero ☐ Falso

Falso

La afirmación es falsa, lo que realiza es la validación del usuario, no la conexión al servidor.

4.4.- Programación de un cliente Telnet.

Telnet (TELEcommunication NETwork) es un **protocolo que permite acceder a otro equipo de la red y administrarlo de forma remota, esto es, como si estuviéramos sentados delante de él.**

El protocolo Telenet está basado en:

- ✓ El modelo cliente/servidor, por lo que su esquema básico de funcionamiento será el típico de esta arquitectura.
- ✓ El servidor escucha las peticiones por el puerto 23.
- ✓ Su funcionamiento es en modo texto.



Este servicio puede resultar muy útil para administrar por ejemplo equipos sin pantalla o teclado, o bien servidores apilados en un rack o que no estén físicamente presentes.

En la actualidad, no se suele utilizar, debido a la poca seguridad que ofrece, ya que toda la información que se intercambia entre servidor y cliente, incluidos usuario y contraseña, se transmiten en texto plano. Esto es así, porque Telnet se creó pensando en la facilidad de uso y no en la seguridad. En su lugar, se utiliza un servicio de acceso y control remoto basado en el protocolo SSH.

Veremos un ejemplo de **programación de un cliente Telnet, sólo a modo ilustrativo de uso de otra biblioteca proporcionada por el API de Apache**, `org.apache.commons.net`, que ya te descargaste en los apartados anteriores.

La biblioteca necesaria es `org.apache.commons.net.telnet`. Para disponer de ella, tendremos que agregar a las bibliotecas del proyecto, como en el ejemplo de FTP, el archivo `commons-net-n.x.x.jar`. Entre las clases que proporciona para programar un cliente Telnet está:

- ✓ **Clase `TelnetClient`.** Permite implementar un terminal virtual para el protocolo Telnet. Hereda de la clase `SocketClient`.
 - El método `SocketClient.connect()` realiza la conexión al servidor.
 - Los métodos `TelnetClient.getInputStream()` y `TelnetClient.getOutputStream()` permiten a través de objetos `InputStream()` y `OutputStream()` enviar y recibir datos a través de la conexión Telnet.
 - El método `TelnetClient.disconnect()` cierra la conexión al servidor, así como los flujos de entrada y salida y el `socket` abiertos.

En el siguiente enlace puedes ver el código de programación de un cliente Telnet. (Puedes utilizar las bibliotecas de Apache para Telnet o bien las indicadas en el ejemplo).

[Ejemplo Java de un cliente Telnet.](#)

Autoevaluación

Señala si la afirmación siguiente es verdadera o falsa:

El método `telnet.connect("Miservidor")` realiza una conexión TelnetTelnet al servidor remoto Miservidor.

☐ Verdadero ☐ Falso

Falso

La afirmación es falsa, falta especificar el puerto 23.

Para saber más

En el siguiente enlace dispones de toda la información sobre la biblioteca del API de Apache para el servicio Telnet.

[Biblioteca de Apache para el servicio Telnet.](#)

Si quieres conocer más sobre el servicio SSH, consulta este enlace:

[Protocolo SSH.](#)

4.5.- Programación de un cliente SMTP.

Vamos a ver dos ejemplos de programación de clientes SMTP. Uno de ellos basado en `sockets` y otro realizado mediante el API `javax.mail`.

En el primer caso, usando `sockets`, el ejemplo completo y explicado en detalle lo puedes consultar en el siguiente enlace:

[Ejemplo de cliente de correo SMTP basado en sockets.](#)



En el segundo caso, utilizaremos el API `javax.mail`. Este paquete proporciona las clases necesarias para implementar un sistema de correo. Vamos a destacar las **clases y métodos del paquete `javax.mail` que nos permitirán crear nuestro cliente de correo**:

- ✔ **Clase `Session`.** Representa una sesión de correo. Agrupa las propiedades y valores por defecto que utiliza el API `javax.mail` para el correo.
 - Método `getDefaultInstance()`. Obtiene la sesión por defecto. Si no ha sido configurada, se creará una nueva de manera predeterminada. El parámetro que se le pasa debe recoger al menos las siguientes propiedades: protocolo y servidor **smtp**, puerto para el `socket` de sesión y tipo, usuario y puerto **smtp**).
- ✔ **Clase `Message`.** Modela un mensaje de correo electrónico.
 - Método `setFrom()`. Asigna el atributo `From` al mensaje, siendo éste la dirección del emisor.
 - Método `setRecipients()`. Asigna el tipo y direcciones de destinatarios.
 - Método `setSubject()`. Para indicar asunto del mensaje.
 - Método `setText()`. Asigna el texto o cuerpo del mensaje.
- ✔ **Clase `Transport`.** Representa el transporte de mensajes. Hereda de la clase `Service`, la cual proporciona funcionalidades comunes a todos los servicios de mensajería, tales como conexión, desconexión, transporte y almacenamiento.
 - Método `send()`. Realiza el envío del mensaje a todas las direcciones indicadas. Si alguna dirección de destino no es válida, se lanza una excepción `SendFailedException`.

En el siguiente documento te indicamos los pasos a seguir para descargar el API `javax.mail` e integrarla en NetBeans.

[API javax.mail. Descarga e integración NetBeans.](#)

[Resumen textual alternativo](#)

En el siguiente enlace dispones del proyecto java completo. Observa que no se realiza de forma explícita una conexión y desconexión, esto es debido a que va implícito en el objeto `Transport`. Cuando ejecutes el programa, recuerda poner los datos reales de tu cuenta de Gmail (o bien crea una) así como un destinatario válido (puede ser tu misma cuenta de Gmail). Observa también que el protocolo SMTP no utiliza el puerto 25, esto es porque se trata de SMTP seguro (SMTP sobre SSL).

[Código del proyecto Java para programar un cliente SMTP.](#) (0.01 MB)

Para saber más

Consulta el siguiente enlace para conocer a muchas otras posibilidades de programación de sistemas de correo con el API `javax.mail`.

[Documentación del API `javax.mail`.](#)

5.- Programación de servidores.

Caso práctico

Hoy **Ada** se ha reunido con **Juan** y **Ana** para ver la marcha del proyecto. **Ana** le ha comentado que van a revisar el servidor web de la aplicación, pues ya saben que no gestiona bien la concurrencia de peticiones, y que además, tarda mucho en procesar algunas de ellas. **Ada** les ha insistido en que además de solucionar el problema de los accesos simultáneos, deben conseguir que todos los servicios programados estén optimizados en cuanto a tiempos de respuesta. **Juan** y **Ana**, la miran y dicen -de acuerdo **Ada**, lo tendremos en cuenta.

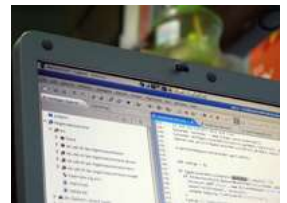


Entre los diferentes aspectos que se deben tener en cuenta cuando se diseña o se programa un servidor o servicio en red, vamos a resaltar los siguientes:

- ✓ El servidor debe poder **atender a multitud de peticiones que pueden ser concurrentes en el tiempo**. Esto lo podemos conseguir mediante la programación del servidor utilizando hilos o Threads.
- ✓ Es importante **optimizar el tiempo de respuesta del servidor**. Esto lo podemos controlar mediante la monitorización de los tiempos de proceso y transmisión del servidor.

La clase `ServerSocket` es la que se utiliza en Java a la hora de crear servidores. Para programar servidores o servicios basados en protocolos del nivel de aplicación, como por ejemplo el protocolo HTTP, será necesario conocer el comportamiento y funcionamiento del protocolo de aplicación en cuestión, y saber que tipo de mensajes intercambia con el cliente ante una solicitud o petición de datos.

En los siguientes apartados vamos a ver el ejemplo de cómo programar un servidor web básico, al que después le añadiremos la funcionalidad de que pueda atender de manera concurrente a varios usuarios, optimizando los recursos.



Para saber más

En el siguiente enlace puedes ver el código completo de la programación del servidor y el cliente de un servicio básico de chat.

[Ejemplo sobre la creación de un Chat.](#)

Citas para pensar

"Sin emoción no hay proyecto."

Eduardo Punset

5.1.- Programación de un servidor HTTP.

Antes de lanzarnos a la programación del servidor HTTP, vamos a recordar o conocer cómo funciona este protocolo y a sentar las hipótesis de trabajo.

El servidor que vamos a programar cumple lo siguiente:

- ✓ Se basa en la versión 1.1 del protocolo HTTP.
- ✓ Implementará solo una parte del protocolo.
- ✓ Se basa en dos tipos de mensajes: peticiones de clientes a servidores y respuestas de servidores a clientes.
- ✓ Nuestro servidor solo implementará peticiones **GET**.



Debes conocer

¿Cómo procesa el servidor web las peticiones del cliente? Dependerá del método utilizado en la petición por el cliente, que en nuestro ejemplo es el **GET**. Consulta la siguiente web para ver cómo se hace.

[Respuesta a una petición HTTP.](#)

Para crear un servidor HTTP o servidor web, el esquema básico a seguir será:

- ✓ Crear un **socketServer** asociado al puerto 80 (puerto por defecto para el protocolo HTTP).
- ✓ Esperar peticiones del cliente.
- ✓ Aceptar la petición del cliente.
- ✓ Procesar petición (intercambio de mensajes según protocolo + transmisión de datos).
- ✓ Cerrar **socket** del cliente.

A continuación, vamos a programar un sencillo servidor web que acepte peticiones por el puerto 8066 de un cliente que será tu propio navegador web. Según la URL que incluyas en el navegador, el servidor contestará con diferente información. Los casos que vamos a contemplar son los siguientes:

- ✓ Al poner en tu navegador **http://localhost:8066**, te dará la bienvenida.
- ✓ Al poner en tu navegador **http://localhost:8066/quijote**, mostrará un párrafo de el Quijote.
- ✓ Al poner en tu navegador una URL diferente a las anteriores, como por ejemplo **http://localhost:8066/a**, mostrara un mensaje de error.

Recuerda detener o parar el servidor, una vez lo hayas probado, antes de volver reiniciarlo.

En el siguiente enlace dispones del proyecto java completo:

[Ejemplo de servidor HTTP.](#) (0,02 MB)

Autoevaluación

Señala si la afirmación siguiente es verdadera o falsa:

El método GET lo utiliza el servidor para enviar información al cliente.

☐ Verdadero ☐ Falso

Falso

La afirmación es falsa, es un método que utiliza el cliente para realizar la petición al servidor.

Para saber más

En el siguiente enlace dispones de la información necesaria para conocer el funcionamiento del protocolo HTTP.

[Funcionamiento del protocolo HTTP.](#)

5.2.- Implementar comunicaciones simultáneas.

Para proporcionar la funcionalidad a nuestro servidor, de que pueda atender comunicaciones simultáneas es necesario utilizar hilos o threads.

Un servidor HTTP realista tendrá que atender varias peticiones simultáneamente. Para ello, tenemos que ser capaces de modificar su código para que pueda utilizar varios hilos de ejecución. Esto se hace de la siguiente manera:

- ✓ El hilo principal (o sea, el que inicia la aplicación) creará el **socket** servidor que permanecerá a la espera de que llegue alguna petición.
- ✓ Cuando se reciba una, la aceptará y le asignará un **socket** cliente para enviarle la respuesta. Pero en lugar de atenderla él mismo, el hilo principal creará un nuevo hilo para que la despache por el **socket** cliente que le asignó. De esta forma, podrá seguir a la espera de nuevas peticiones.

Esquemáticamente, **el código del hilo principal tendrá el siguiente aspecto:**

```
try {
    socServidor = new ServerSocket(puerto);
    while (true) {
        //acepta una petición, y le asigna un socket cliente para la respuesta
        socketCliente = socServidor.accept();
        //crea un nuevo hilo para despacharla por el socket cliente que le asignó
        hilo = new HiloDespachador(socketCliente);
        hilo.start();
    }
} catch (IOException ex) {
}
```

donde la clase **HiloDespachador** será una extensión de la clase **Thread** de Java, cuyo constructor almacenará el **socket cliente** que recibe en una variable local utilizada luego por su método **run()** para tramitar la respuesta:

```
class HiloDespachador extends Thread {
    private socketCliente;
    public HiloDespachador(Socket socketCliente) {
        this.socketCliente = socketCliente;
    }
    public void run() {
        try{
            //tramita la petición por el socket cliente
        } catch (IOException ex) {
        }
    }
}
```

Hecho esto, tendremos todo un Servidor HTTP capaz de gestionar peticiones concurrentes de manera más eficiente.

Para saber más

Aquí tienes un ejemplo mucho más avanzado de un servido HTTP completo que usa un pool de hilos para gestionar la concurrencia:

[Servidor web Multihilo.](#)

En este otro enlace puedes consultar la clase **Thread** de Java

[Clase Thread de Java.](#)



5.3.- Monitorización de tiempos de respuesta.

Una cuestión muy importante para evaluar el rendimiento y comportamiento de una aplicación cliente/servidor es monitorizar los tiempos de respuesta del servidor. Los tiempos que intervienen desde que un cliente realiza una petición al servidor hasta que recibe su resultado son dos:

- ✔ **Tiempo de procesamiento.** Es el tiempo que el servidor necesita para procesar la petición del cliente y enviar los datos.
- ✔ **Tiempo de transmisión.** Es el tiempo que tardan los mensajes en llegar a su destino a través de la red.



¿Cómo podemos medir esos tiempos?

Para **medir el tiempo de procesamiento** basta medir el tiempo que transcurre en el servidor para procesar la solicitud del cliente. Por ejemplo, el siguiente código permite medir ese tiempo en milisegundos:

```
// Control de tiempo del inicio del tiempo desde se recibe el procesamiento
long tiempoInicio = System.currentTimeMillis()
// Procesamiento de la petición del cliente
// Control de tiempo del final del tiempo al finalizar el procesamiento
long tiempoFin = System.currentTimeMillis()
long tiempoProcesamiento = tiempoFin - tiempoInicio
System.out.println("Tiempo de proceso: " + tiempoProcesamiento + " ms")
```

[Código Java ejemplo para medir tiempo de procesamiento.](#) (0,01 MB)

Para **medir el tiempo de transmisión** será necesario que el servidor envíe un mensaje con el tiempo del sistema al cliente. El cliente al recibir el mensaje debe calcular su tiempo de sistema y compararlo con el tiempo recibido en el mensaje.

Para poder comparar los tiempos de respuesta de dos equipos es imprescindible que sus relojes de sistema estén sincronizados a través de cualquier servicio de tiempo (NTP). En equipos Windows por ejemplo, la sincronización de los relojes se realiza automáticamente.

Autoevaluación

Señala la opción correcta. Para monitorizar el tiempo de respuesta del servidor:

- ☐ Basta con controlar el tiempo de proceso del servidor.
- ☐ Solo interesa realizar esta tarea si hay sobrecarga.
- ☐ Solo depende del tiempo de transmisión.
- ☐ Se debe tener en cuenta el tiempo de proceso del servidor y el tiempo de transmisión.

No es correcto, prueba de nuevo.

Incorrecto, deberías haber leído mejor.

No es cierto, debes prestar más atención.

Muy bien, vamos por buen camino.

Solución

1. Incorrecto
2. Incorrecto
3. Incorrecto
4. Opción correcta

Para saber más

Consulta el siguiente enlace para obtener más información sobre el servicio NTP.

[El servicio NTP.](#)

5.4.- Ejemplo de monitorización del tiempo de transmisión.

Vamos a ver un ejemplo de cómo calcular el tiempo de transmisión de un recurso desde un servidor web. Para ello, vamos a modificar el cliente HTTP realizado anteriormente con las clases `URL` y `URLConnection`, para añadir la función `URLConnection.getDate()` que proporciona el instante en el que el servidor inicia la transmisión de su respuesta al cliente. Ese tiempo es enviado por el servidor al cliente mediante la cabecera `Date`, al igual que la cabecera que vimos anteriormente sobre el tipo de contenido, y que se podía recuperar con el método `URLConnection.getContentType()`.

Desde el siguiente enlace puedes descargar el proyecto Java completo.

[Ejemplo de monitorización de tiempo de transmisión.](#) (0.02 MB)



Citas para pensar

"Lo que tenemos que aprender, lo aprendemos haciendo."

Aristóteles

Para saber más

Mediante los comandos `PING` y `TRACER` puedes medir el tiempo que tarda un servidor en responder a tu petición. Consulta el siguiente enlace para más información.

[Medir el tiempo de respuesta de un servidor.](#)

Anexo.- Licencias de recursos.

Licencias de recursos utilizados en la Unidad de Trabajo.

Recurso (1)	Datos del recurso (1)	Recurso (2)	Datos del recurso (2)
	Autoría: Manfred Wassmann. Licencia: CC BY. Procedencia: http://www.flickr.com/photos/berlinframes/5820431287/		Autoría: Gabiwxp. Licencia: CC BY-SA. Procedencia: http://es.wikipedia.org/wiki/Archivo:Activo.svg
	Autoría: Paul Downey. Licencia: CC BY 2.0. Procedencia: http://www.flickr.com/photos/psd/263092176/		Autoría: Hillary h. Licencia: CC BY-SA. Procedencia: http://www.flickr.com/photos/hillaryandanna/12
	Autoría: Gov/Ba. Licencia: CC-by. Procedencia: http://www.flickr.com/photos/agecombahia/4479505820/in/photostream/		Autoría: Ben Sheldon. Licencia: CC BY-NC-SA. Procedencia: http://www.flickr.com/photos/bensheldon/2121/
	Autoría: Javier RiGa. Licencia: CC BY . Procedencia: http://www.flickr.com/photos/74291475@N05/6833896015/		Autoría: RonLD. Licencia: CC-BY. Procedencia: http://www.flickr.com/photos/rld/1204516152
	Autoría: Luis Pérez. Licencia: CC BY-NC-SA. Procedencia: http://www.flickr.com/photos/luipermom/2934628860/		Autoría: Peter Smile. Licencia: CC BY-NC-SA 2.0. Procedencia: http://www.flickr.com/photos/petersmile/24481/
	Autoría: Francisco J. Gámez. Licencia: CC BY-NC-SA 2.0. Procedencia: http://www.flickr.com/photos/franciscojosegomez/6026289407/		